IMDb MOVIE REVIEW SENTIMENT ANALYSIS

Full Technical Project Report

Databricks • PyTorch • DistilBERT • Hugging Face • Streamlit

Project detail	Recorded value
Project type	End-to-end NLP / binary sentiment classification
Dataset	IMDb 50K Movie Reviews (Stanford NLP / Hugging Face)
Model	DistilBERT Base Uncased, fine-tuned for two classes
Test accuracy	91.69%
Deployment	Hugging Face Model Hub + Streamlit Community Cloud
Report date	30 October 2026

Prepared as a technical record of the data pipeline, model development, evaluation, inference tests and deployment.

Executive Summary

This project delivers an end-to-end movie-review sentiment classifier. IMDb review data was prepared in Databricks using a Bronze–Silver–Gold Medallion Architecture. The Gold data was exported to Google Colab, where DistilBERT was fine-tuned with PyTorch. The model was evaluated on 25,000 held-out reviews, hosted on Hugging Face, and connected to a Streamlit application deployed through Streamlit Community Cloud.

Headline result: 91.69% test accuracy and 91.79% positive-class F1-score. The model correctly classified 22,922 of 25,000 reviews. Positive precision was 90.72% and positive recall was 92.87%.

- The original 25,000 training records became 24,877 Gold records after removing 123 exact text overlaps with the test set. All 25,000 test records were retained.
- DistilBERT was fine-tuned for two epochs, with maximum sequence length 256, batch size 8, gradient accumulation 2 and FP16 mixed precision on a Tesla T4 GPU.
- The model and tokenizer were hosted in a public Hugging Face repository; GitHub contains the application code and documentation, not the 255 MB model weights.

The main limitations are 256-token truncation, evaluation on one held-out test set, CPU inference in the hosted app, and a small qualitative error sample.

Model repository: SHAILESHGOKU/imdb-distilbert-sentiment. Add the exact Streamlit app URL and GitHub repository URL to the final README.

1. Introduction

Movie reviews contain subjective opinions expressed in natural language. Sentiment analysis classifies text polarity, commonly as positive or negative. This project builds a binary classifier for IMDb movie reviews and exposes it through a browser-based application.

1.1 Objectives

- Prepare IMDb train and test reviews through a Databricks Medallion pipeline.
- Identify and remove exact text overlap from training while preserving the held-out test set.
- Fine-tune DistilBERT for two-class sentiment classification.
- Evaluate accuracy, precision, recall, F1-score, loss and confusion matrix.
- Save the model and tokenizer for independent inference.
- Deploy a Streamlit interface that accepts raw review text and returns sentiment and class probabilities.

1.2 Scope

The implemented task is binary review-level sentiment classification. It does not classify emotions, sentiment intensity, movie genre or specific review aspects such as acting and cinematography.

2. Dataset and Exploratory Analysis

Dataset: stanfordnlp/imdb. The standard labelled train and test splits each contained 25,000 reviews. The unsupervised split was not used. Label 0 represented Negative and label 1 represented Positive.

Dataset component	Rows	Purpose
Original train	25,000	Initial model fitting source
Original test	25,000	Held-out evaluation
Gold train	24,877	Training after overlap removal
Gold test	25,000	Test set retained unchanged

The original training split was balanced at 12,500 examples per class. After deduplication, Gold training contained 12,405 negative and 12,472 positive reviews. The test set remained balanced at 12,500 per class.

2.1 Review length and tokenization EDA

EDA recorded mean token length of approximately 313.87, median 233, 75th percentile 382, 95th percentile approximately 806 and maximum 3,127 tokens. At a 256-token maximum, 44.25% of reviews exceeded the limit; at 512 tokens, 14.8% exceeded it. The 256-token setting was selected as a practical compute trade-off for a free Colab GPU.

2.2 Data quality and overlap check

- There were 123 exact text overlaps between original train and test; labels agreed for all (95 negative and 28 positive).
- Overlapping texts were removed from training only. The test set remained unchanged; shared unique text count after Gold construction was zero.
- Final CSV exports loaded with shapes (24,877, 3) and (25,000, 3). Null values, empty reviews and invalid labels were all zero.
- Cleaning replaced HTML line-break markup such as <br /> with spaces and trimmed whitespace. Punctuation and stopwords were not aggressively removed.

3. Databricks Medallion Data Pipeline

Databricks organized data preparation into Bronze, Silver and Gold layers, separating raw ingestion, standardized text preparation and model-ready dataset construction.

Layer	Implementation	Output
Bronze	Ingested original train/test; retained text and label; added ingestion timestamp.	bronze.train_reviews (25,000); bronze.test_reviews (25,000)
Silver	Replaced HTML line breaks, trimmed text, created clean_text and text_length; validated data.	silver.train_reviews (25,000); silver.test_reviews (25,000)
Gold	Identified exact clean_text overlap; removed overlapping training rows using left anti join; retained test rows.	gold.train_reviews (24,877); gold.test_reviews (25,000)

3.1 Bronze

Bronze tables were created in catalog imdb_sentiment under schema bronze. Original review text and labels were retained, with an ingestion timestamp added for basic lineage.

3.2 Silver

Silver replaced HTML line-break markup, trimmed whitespace and created clean_text and text_length. Validation reported zero nulls, zero empty reviews, zero invalid labels and zero remaining line-break tags.

3.3 Gold

Distinct clean_text values from train and test were joined to identify overlap. Overlapping training rows were removed with a left anti join; test data was not modified. Validation confirmed no shared unique text remained.

3.4 Export to Colab

Gold tables were exported through a Unity Catalog Volume because Databricks Free Edition disabled the public DBFS root. Parquet loading in Colab encountered Arrow parsing errors, so CSV exports with UTF-8 encoding and explicit quote escaping were used. Final files were loaded as train_v2.csv and test_v2.csv.

4. Model Development

4.1 Model architecture

The model was **distilbert/distilbert-base-uncased**, loaded as AutoModelForSequenceClassification with two labels. Recorded configuration: hidden size 768, six Transformer layers, 12 attention heads and 66,955,010 parameters. The classification head was newly initialized and learned during fine-tuning; unused pretraining-head weights were expected when adapting the base checkpoint.

4.2 Tokenization and input pipeline

The DistilBERT tokenizer converted raw text into WordPiece token IDs and attention masks, adding special tokens. Inputs were truncated at 256 tokens. Dynamic padding was applied per batch with DataCollatorWithPadding. A PyTorch Dataset wrapper returned input_ids, attention_mask and labels. Test text was tokenized with the same tokenizer but not used for gradient updates.

4.3 Training configuration

Setting	Value
Checkpoint	distilbert/distilbert-base-uncased
Task	Binary sequence classification
Maximum sequence length	256
Batch size	8
Gradient accumulation	2 (effective batch size 16)
Epochs	2
Learning rate	2e-5

Setting	Value
Weight decay	0.01
Optimizer	AdamW
Scheduler	Linear decay; 10% warmup
Mixed precision	FP16 (torch.amp autocast + GradScaler)
Training hardware	Google Colab NVIDIA Tesla T4 GPU

The scheduler was configured for 3,110 optimizer steps across two epochs, including 311 warmup steps. Epoch checkpoints were saved, followed by the final model and tokenizer in /content/imdb_distilbert_final.

5. Evaluation Results

The model was evaluated on all 25,000 Gold test reviews in evaluation mode with gradients disabled. The predicted class was selected from the two output logits.

5.1 Training loss

Epoch	Training loss	Approx. elapsed time
1	0.3014	3 min 10 sec
2	0.1534	3 min 11 sec

Training loss declined from 0.3014 to 0.1534, approximately a 49.1% reduction. This describes the training batches; held-out test metrics are the separate evaluation measure.

5.2 Test metrics

Metric	Result	Interpretation
Test loss	0.2213	Average evaluation loss reported by the model
Accuracy	91.69%	Correct predictions among all test reviews
Positive precision	90.72%	Share of positive predictions that were actually positive
Positive recall	92.87%	Share of actual positive reviews identified
Positive F1	91.79%	Harmonic mean of positive precision and recall

Figure 1. Accuracy is overall; precision, recall and F1 are for the positive class.

Negative-class precision was 0.93, recall 0.91 and F1 0.92. Macro and weighted averages were both approximately 0.92, consistent with the balanced test set.

5.3 Confusion matrix

Actual / predicted	Predicted negative	Predicted positive
Actual negative	11,313 (TN)	1,187 (FP)
Actual positive	891 (FN)	11,609 (TP)

The model correctly classified 22,922 reviews and incorrectly classified 2,078. False positives (1,187) were more frequent than false negatives (891). Negative recall was 90.50%; positive recall was 92.87%.

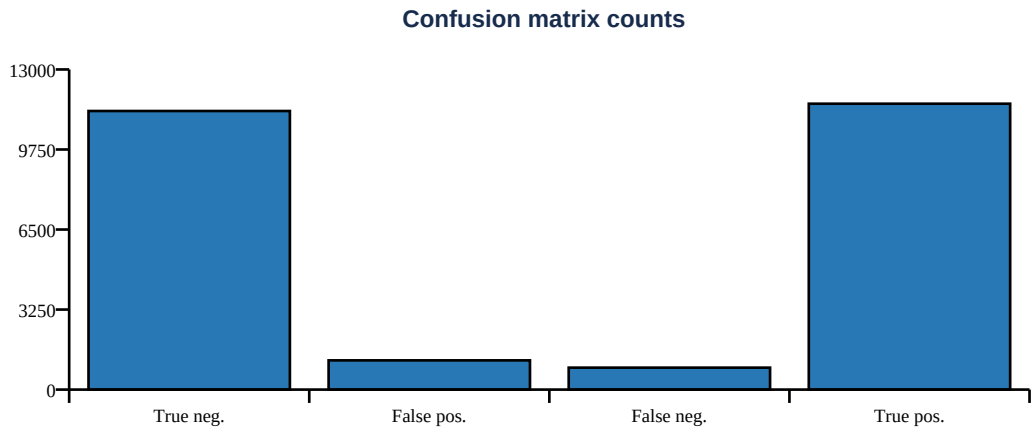

Figure 2. Confusion-matrix counts for the 25,000-review test set.

5.4 Interpretation

- The model performed strongly on this balanced IMDb test set, with 91.69% overall accuracy.
- Positive recall exceeded positive precision, so the model identified a high share of positive reviews but also made false-positive predictions.
- False positives exceeded false negatives by 296 examples in this test sample.
- Results are specific to this dataset, preprocessing, sequence length and training run; they should not be generalized to other text domains without evaluation.

6. Qualitative Error Analysis

Ten misclassified reviews were sampled and inspected. Patterns included mixed sentiment, sarcasm or irony, context-dependent wording, difficulty aggregating multiple opinions and possible label ambiguity. Examples included reviews praising an actor while criticizing direction, or praising visuals while criticizing commentary. Some positive-labelled examples contained extensive negative language.

This was a qualitative inspection of ten examples, not a formal annotation study. It does not establish how frequently each pattern occurs across all 2,078 errors. Some apparent errors may reflect subjective or inconsistent dataset labels.

A proposed follow-up was to compare error rates across review-length bands. That analysis was not completed; therefore this report does not claim that truncation caused the observed errors.

7. Model Saving and Inference

7.1 Model packaging

File	Purpose
config.json	Model architecture and configuration
model.safetensors	Fine-tuned model weights
tokenizer.json	Tokenizer vocabulary and tokenization data
tokenizer_config.json	Tokenizer configuration

The final model and tokenizer were saved in /content/imdb_distilbert_final. During inference testing, a device mismatch occurred because the model was on CPU while inputs were sent to CUDA. It was resolved by explicitly moving the model to cuda:0 and confirming that model and input tensors shared the same device.

7.2 Custom inference checks

Test input	Prediction	Softmax confidence
Clearly positive custom review	Positive	99.66%
Clearly negative custom review	Negative	99.56%

Both hand-written examples returned the expected polarity. These checks verified tokenization, model execution, softmax and output display. Two examples are not a statistically meaningful evaluation set. Softmax confidence is not a guarantee of correctness or a calibrated probability.

8. Streamlit Application and Deployment

8.1 User interface

The Streamlit app provides a review text area and an Analyze Sentiment button. Results include Positive or Negative sentiment, predicted-class confidence and both class probabilities. An About section describes the model and reported evaluation metrics.

The application loads the model and tokenizer from Hugging Face and caches them with Streamlit's resource cache. CPU inference is used for compatibility with Streamlit Community Cloud, where GPU availability is not assumed.

8.2 Hosting architecture

Platform	Responsibility
GitHub	Stores app.py, requirements.txt and README.md
Hugging Face Model Hub	Hosts model configuration, weights and tokenizer files
Streamlit Community Cloud	Installs dependencies, downloads model and runs the web app

Model repository: SHAILESHGOKU/imdb-distilbert-sentiment. The 255 MB model weights were excluded from GitHub because the file exceeds GitHub's standard 100 MB per-file limit. Streamlit downloads the model from Hugging Face at startup.

Deployment status: the project owner confirmed that Streamlit Community Cloud deployment completed successfully. The exact live application URL was not supplied; add the verified URL to the GitHub README and portfolio.

9. Reproducibility and Repository

Component	Recorded version / environment
Databricks Spark	4.2.0
Google Colab Python	3.13 (reported later in workflow)
PyTorch	2.11.0+cu130
Transformers	5.20.2
Streamlit	1.66.1
Training GPU	NVIDIA Tesla T4
Model checkpoint	distilbert/distilbert-base-uncased

The local Windows upload environment used Python 3.14 and huggingface_hub 2.4.0. This was separate from Colab training and the Streamlit Cloud runtime.

```
imdb-sentiment-analyzer/
■■■ app.py
■■■ requirements.txt
■■■ README.md
```

The application references the public model ID SHAILESHGOKU/imdb-distilbert-sentiment, truncates input at 256 tokens, runs CPU inference, applies softmax and displays the prediction and probabilities.

10. Limitations and Risks

Limitation	Implication
256-token maximum	Long reviews are truncated; later context may be omitted.
IMDb-specific training	Performance on other domains is unknown.
Single held-out evaluation	Use a validation split from training for future model selection; do not repeatedly tune on test.
Small qualitative sample	Ten inspected errors cannot establish prevalence of error causes.
Softmax confidence	Not necessarily calibrated; not a guarantee of correctness.

Limitation	Implication
CPU hosting	Inference may be slower than the Colab T4 GPU.
Public model repository	Model is downloadable; never upload secrets or private data.
Cloud dependency changes	Pin compatible dependencies and inspect logs after environment changes.

This project is a learning and demonstration application, not a high-stakes decision system. Predictions should not be the sole basis for consequential decisions.

11. Future Improvements

The following are proposed extensions, not completed features.

11.1 Model and evaluation

- Create a validation split from Gold training data for controlled experiments while reserving the current test set for final assessment.
- Compare sequence lengths 256, 384 and 512, measuring quality, memory use and runtime.
- Assess calibration using reliability diagrams, Brier score or expected calibration error.
- Inspect a larger stratified sample of false positives and false negatives using a consistent annotation rubric.
- Compare DistilBERT with a TF-IDF plus logistic-regression baseline.

11.2 Application and operations

- Add sample review buttons and a visible warning when input is truncated.
- Add session-only prediction history if useful, without retaining submitted reviews by default.
- Document the privacy behavior of user-submitted text.
- Monitor model download failures, startup time and inference latency.
- Record dataset and model version metadata for reproducibility.

11.3 Data engineering and analytics

- Automate ingestion, validation and Gold exports.
- Build a Databricks dashboard for data quality, class balance, review length and pipeline record counts.
- Track anonymized aggregate prediction counts only with an explicit privacy and retention policy.

12. Conclusion

The IMDb Sentiment Analysis project successfully combined data engineering, natural language processing, deep learning and application deployment. Databricks Medallion Architecture produced validated, deduplicated training data. DistilBERT was fine-tuned in Google Colab using PyTorch and achieved 91.69% accuracy and 91.79% positive-class F1-score on 25,000 held-out reviews. The saved model was hosted on Hugging Face and integrated into a Streamlit application deployed through Streamlit Community Cloud.

The main achievement is the complete workflow from raw data preparation to a usable web application. The next phase should focus on evaluation discipline, the effect of long-review truncation, hosted inference performance and portfolio documentation—not accuracy improvement alone.

Appendix A — Key Artifacts

Artifact	Location / notes
Dataset	stanfordnlp/imdb on Hugging Face Datasets
Gold train table	imdb_sentiment.gold.train_reviews — 24,877 rows
Gold test table	imdb_sentiment.gold.test_reviews — 25,000 rows
Final local model	/content/imdb_distilbert_final
Hugging Face model	https://huggingface.co/SHAILESHGOKU/imdb-distilbert-sentiment
GitHub repository	Insert actual repository URL
Live Streamlit app	Insert exact deployed URL

Appendix B — Metric definitions

- Accuracy = (true positives + true negatives) / all predictions.
- Positive precision = true positives / all positive predictions.

- Positive recall = true positives / all actual positive examples.
- Positive F1 = harmonic mean of positive-class precision and recall.
- False positive = an actual negative review predicted as positive.
- False negative = an actual positive review predicted as negative.

Report provenance

This report is based on project outputs and implementation details recorded during the project conversation. It does not independently rerun notebooks or inspect the current live application or repositories. The exact GitHub and live Streamlit URLs were not supplied and are left as fields to complete. Metrics and configuration values are reported as recorded; no unobserved experiments or results have been added.